--

layout: two-cols

Data Quality vs. Validation

Concept 1 of 3

Data Quality

A **continuous property** — how fit the data is for its purpose.

Dimension	Question
Completeness	Are all records present?
Accuracy	Does the value reflect reality?
Consistency	Same fact, same meaning everywhere?
Timeliness	Fresh enough to act on?
Uniqueness	No duplication distorting metrics?

::right::

Data Validation A **point-in-time gate** — does this data conform to a rule right now? Binary: pass or fail. |Check |Validates | |---|---| |**Structural** |Column present, right type? | |**Null check** |Value not null? | |**Uniqueness** |Key not duplicated? | |**Referential**|FK exists in referenced table?| |**Domain** |Value in allowed set? |

---

layout: two-cols

Two Failure Surfaces

Concept 2 of 3

🔴 Structural Failure

The **shape** of the model is wrong — regardless of values inside it.

- A column disappears from SELECT
- Type changes: `TIMESTAMP_NTZ` → `TIMESTAMP_LTZ`
- A HubSpot field is renamed upstream
- A cast is silently removed in a refactor

Caught by: Contracts

::right::

🟢 Data Quality Failure The **values** inside the model are wrong — even if structure is correct. - Duplicate surrogate keys from a merge bug - FK references a deleted dimension record - Wrong SLA bucket from bad CASE WHEN logic - SCD2 has two current rows for the same contact
Caught by: Tests
Perfect structure
+ terrible data
contracts pass, tests fail
Correct data
+ broken schema
tests pass, contracts fail
Both correct
= trustworthy model
all checks pass ✓

---

layout: two-cols

Where Problems Originate

Concept 3 of 3

🟠 **Origin 1 — Upstream API**
HubSpot renames a field. Bronze Lambda picks up the new name silently. Silver still references the old name.
`column disappears` · `type changes`

🟣 **Origin 2 — Inside dbt**
Developer refactors a CTE, removes a cast, or accidentally drops a column from the final SELECT.
`type cast removed` · `column dropped`

🟢 **Origin 3 — Logic / Data**
SCD2 merge bug, wrong CASE WHEN, orphaned FK after a dim record is deleted.
`bad values` · `duplicates` · `orphaned FKs`

::right::

Which Tool Catches What |Problem |Tool | |---|---| |HubSpot field rename |**Contract** | |Bronze Lambda drops column |**Contract** | |Column dropped in refactor |**Contract** | |Type cast changed |**Contract** | |Duplicate surrogate keys |**Test: unique** | |NULL in required column |**Test: not_null** | |Orphaned FK |**Test: relationships** | |Wrong CASE WHEN |**Test: singular** | |Two current SCD2 rows |**Test: singular** | |Value passes rule but wrong| ⚠️ Neither — monitoring gap|

---

layout: two-cols

Contracts

dbt Tools 1 of 5

A **compile-time structural guarantee**. Validates the model's SELECT output against the declaration in `schema.yml` before a single row is written.

Execution timing

```
dbt run
  ├─ [compile]   ← CONTRACT fires here
  │               fails before any write
  ├─ [materialize]   model writes to Snowflake
  └─ [post-hook]     side effects
```

Catches ✓

- Column missing from SELECT
- Wrong type: `TIMESTAMP_LTZ` instead of `TIMESTAMP_NTZ`
- `not_null` column receiving NULLs

Does NOT catch ✗

Duplicate values · bad data values · FK integrity violations

::right::

Example — type problem caught

```
- name: created_at
  data_type: timestamp_ntz
  constraints:
    - type: not_null
    - type: primary_key
      expression: "rely"
```

```
Error: column "created_at"
expected TIMESTAMP_NTZ
but found TIMESTAMP_LTZ
```

Key Snowflake facts - `primary_key` contract **generates real DDL** — no post-hook needed for PK - Add `expression: "rely"` → Snowflake query optimizer + Cortex AI - `foreign_key` DDL generation ** ⚠️ unconfirmed** — use post-hook as fallback - Contracts only work on **table / incremental** materializations ### Toggle Single-line per model. Safe to disable at any time. Enable only when schema is stable **60+ days**.

---

dbt Test Types

dbt Tools 2 of 5

Built-in

unique

Every PK. Always error. A duplicate PK double-counts in every downstream aggregation.

not_null

Every PK, every FK, every critical column. NULLs in PKs break joins silently.

accepted_values

Stable enums. error if frozen, warn if evolving.

relationships

Every FK. Orphaned FKs produce invisible NULLs in star schema joins.

→ Prod

Packages

expression_is_true

Cross-column rules. resolved_at > created_at. Built-in can't compare two columns.

unique_combination_of_columns

Composite PKs. Built-in unique only handles single columns.

expect_column_between

SLA days 0–730. Catches date arithmetic overflows.

expect_row_count_between

Fact table floor. Merge bug emptying table caught here before Power BI loads.

→ Prod

Singular

SQL in /tests — rows returned = failure

One current SCD2 row

Merge bug creates two is_current = true rows for same HubSpot ID. Downstream always picks wrong one.

No future timestamps

HubSpot parsing bugs produce 2099. Breaks Power BI time intelligence.

Referential with tolerance

Flexible FK check with WHERE filters or percentage threshold.

→ Prod

Generic (macro)

Reusable logic in /macros — called from YAML like built-ins

not_null_where

not_null only for current SCD2 rows. Historical rows may legitimately be NULL.

at_least_n_rows

Row count floor reused across all fact tables. No copy-paste SQL.

no_nulls_in_columns

Bulk not_null across 10+ columns. One line in YAML.

→ Prod

Unit Tests

dbt Core 1.8+ — mock input → expected output

What they are

Mock input rows + expected output. dbt runs model SQL against mocks and compares. No real data needed.

When to write

Complex CASE/WHEN, SLA buckets, date spine logic. Only after a bug has already bitten you.

Prod vs Dev

Dev/CI only. Mock data, not prod data. Tests logic at build time, not runtime.

→ Dev / CI only

---

layout: two-cols

Tests in Practice

dbt Tools 3 of 5

By Layer

Bronze

- Source freshness: warn 6h, error 24h
- `not_null` on PK only
- No uniqueness — duplicates expected in raw

Silver *(Kimball source of truth)*

- `unique` + `not_null` on every surrogate key → **error**
- `relationships` on every FK → **error**
- `accepted_values` on stable enums
- SCD2 singular: one current row per business key

Gold

- `unique` + `not_null` on PK → **error**
- `not_null` on business-critical measure columns
- Row count lower bound on fact tables
- **Net-new only** — don't repeat Silver tests

::right::

Severity Guide |Test |Severity | |---|---| |PK `unique` + `not_null` | 🔴 error | |FK `relationships` | 🔴 error | |Business-critical `not_null` Gold| 🔴 error | |`accepted_values` frozen enums | 🔴 error | |`accepted_values` evolving enums | 🟡 warn | |Optional columns | 🟡 warn | |Row count anomaly | 🟡 warn_if| |Source freshness | 🔴 / 🟡 | ### Net-New Principle

```
Silver tests contact_key → unique, not_null
Gold SELECTs it unchanged  → SKIP in Gold

Gold derives sla_bucket via CASE WHEN
→ new surface → TEST in Gold

Gold adds LEFT JOIN / COALESCE
→ new failure surface → RE-TEST
```

---

layout: two-cols

Post-Hooks

dbt Tools 4 of 5
SQL that executes against Snowflake **after** a model materializes. Every run. No knowledge of schema or data content.

Execution timing

```
dbt run
  ├─ [compile]       Contract fires (if enforced)
  ├─ [materialize]   Model writes to Snowflake
  └─ [post-hook] ←   Fires here — every run
```

⚠️ Important

Post-hooks have **no knowledge** of schema or data. They cannot validate, check, or block anything. They execute arbitrary SQL.

NOT for

- Schema validation
- Data quality checks
- Role grants → manage in **Snowflake RBAC**

::right::

Use cases **PK constraint** *(if no contract)*

```
ALTER TABLE {{ this }}
  ADD PRIMARY KEY (contact_key) RELY;
```

Snowflake metadata only — not enforced. `RELY` enables query optimizer + Cortex AI. **FK constraint**

```
ALTER TABLE {{ this }}
  ADD FOREIGN KEY (owner_key)
  REFERENCES {{ ref('dim_owner') }}(owner_key) RELY;
```

⚠️ Required until contract FK DDL is confirmed. **PII tagging**

```
ALTER TABLE {{ this }}
  SET TAG governance.pii = 'true';
```

Clustering key

```
ALTER TABLE {{ this }}
  CLUSTER BY (created_date);
```

---

layout: two-cols

All Three Together

dbt Tools 5 of 5

Execution Timeline

```
dbt run
  │
  ├─ [compile]      🟠 CONTRACT
  │                 Shape + types validated
  │                 Fails before any write
  │
  ├─ [materialize]  Model writes to Snowflake
  │
  └─ [post-hook]    🟣 POST-HOOK
                    PK/FK DDL, tags, clustering

dbt test  (separate command)
  └─ [test]         🟢 TESTS
                    Data values inspected
                    Bad data may already be in table
```

The non-negotiable rule

Snowflake **does not enforce** PK, FK, or UNIQUE constraints. Ever.
You can insert duplicate PKs and broken FKs — Snowflake accepts all of it silently.

::right::

Summary |Tool |Role |When |Catches | |----------|----------|----------|----------| |**Contract** |Structural guard |Compile |Schema drift| |**Post-Hook**|Snowflake metadata|After write |Nothing | |**Tests** |Data validation |Separate run|Bad values |
The core equation

```
Snowflake constraints
  → describe what data SHOULD be
  → power optimizer + Cortex AI
  → not enforced
```

```
dbt tests
  → verify what data ACTUALLY is
  → the only real enforcement
```

Tests are non-negotiable regardless of which scenario you use — contracts or not.

---

layout: center

Questions?

🟠
Contracts
Structural guard at compile time
🟣
Post-Hooks
Snowflake metadata after write
🟢
Tests
Data validation after materialization
Reference: `dbt_quality_guardrails.md` — full decision matrix, three-step governance plan, model scoring